MANIPAL UNIVERSITY JAIPUR

Department of Computer Science and Engineering

SOFTWARE ENGINEERING LAB (CSE3132)

Project Report

TeamSpark

Hackathon Project Team Formation System

Submitted by

2427030421 Bhavya Bansal

2427030683 Raj Shukla

Course Coordinator: Dr. Bali Devi

V Semester, B.Tech (CSE), July to November 2026

October 2026

Abstract

Students who want to join a hackathon usually form teams through friends or group chats. This leaves many students without a team, and teams often end up with the same skills repeated and important skills missing. TeamSpark is a web-based system that tackles this. A student posts a team listing with the skills it needs. The system ranks students against that listing using a Compatibility Score and shows the top three, with the reasons behind each score. The owner then decides whom to invite. Other students can search listings by skill and send a request to join.

This report covers the full path from problem to tested prototype: the process model, the requirements from the SRS, UML diagrams, data flow and data diagrams, the scoring algorithm, the working website, quality assurance and DevOps practice, and a testing chapter. The testing chapter applies black-box techniques (equivalence partitioning, boundary value analysis, decision tables, state transition testing and scenario testing) and white-box techniques (control flow graphs, decision-to-decision path graphs, cyclomatic complexity, basis path testing and coverage measurement). In total 20 automated test cases were written using Jest and Playwright, and 20 of them pass. The logic tests cover 85.89% of statements and 79.64% of branches of the application code.

The prototype runs in a browser as a single-user demonstration. Login, notifications, administrator features and a shared database belong to the full system and are described as future work.

Contents

1. Introduction..... 4
2. Process Model..... 5
3. Requirements..... 6
4. System Design..... 8
5. Implementation..... 17
6. Quality Assurance and DevOps..... 21
7. Testing..... 22
8. Conclusion and Future Work..... 27
References..... 28
Appendix A: Test Case Table..... 29
Appendix B: Running the Project and Tests..... 31

1. Introduction

1.1 Background and problem

Hackathons are short events in which teams build a working project in one or two days. The quality of a team matters as much as the idea: a team with three front-end developers and nobody who can build a back end will struggle. In colleges, teams are usually formed informally. Students ask friends, post in group chats, or end up with whoever is left. The result is that capable students are left out, teams are unbalanced, and organisers or team leaders spend time on manual matching.

The problem we address is: **how can a student who has a project idea find teammates whose skills fit it, and how can a student looking for a team find the right one, without relying on personal contacts?**

1.2 Objectives

- Let a student post a team listing with a title, description, required skills and team size.
- Let students search open listings by skill and send a request to join.
- Rank students for a listing automatically using a Compatibility Score, and show the reasons behind each score.
- Keep a human in control: the system suggests, the listing owner decides who joins.
- Specify, design, build and test the system following the software engineering process taught in the course.

1.3 Scope

In scope (built as a working prototype): browsing and searching teams, posting and deleting a team, profile with skills and levels, ranked candidate suggestions with an explanation, join requests with accept and reject, and saving data in the browser.

Described in the SRS but not built in the prototype: registration and login, listing edit and close, automatic closing when a team is full, notifications, and the administrator functions (manage listings, manage users, reports). These need a server and a shared database. Chapter 3 lists the status of every requirement.

1.4 Approach used for automated team selection

The course brief asked for automated team selection with attention to the quality of the team. We compared several approaches and chose a **hybrid** method: the algorithm produces a ranked shortlist, and a person makes the final decision. A fully automatic assignment was rejected because a score cannot see how people work together, and because students are more likely to trust a system that explains its suggestions and leaves the decision to them.

1.5 Structure of this report

Chapters 2 to 8 follow the project from process model to conclusion, with the testing in Chapter 7. The full test case table and the run instructions are in the appendices.

2. Process Model

2.1 Choice of model

We used an **agile, incremental** approach. The team has two members, the requirements were refined as we learned more (for example the scoring factors changed once we questioned them), and a working demonstration was needed early so that the idea could be judged. A strict waterfall model would have fixed the requirements too early. A full Scrum setup with roles and ceremonies would be heavier than a two-person project needs, so we used its core ideas only: short iterations, a working increment at the end of each one, and review before moving on.

2.2 Iterations

Iteration	Focus	Output
1	Problem and requirements	Problem statement, SRS introduction, functional and non-functional requirements
2	Design	Six UML diagrams, selection of the hybrid method, Compatibility Score design
3	Prototype	Working website: listings, search, suggestions, requests, profile
4	Testing and review	Automated tests, white-box and black-box analysis, defect fixes, this report

2.3 Why feedback mattered

Two examples show the value of iterating. First, the original score used interest overlap and availability. On review we saw that availability is already shown by a student sending a request, and that interest largely repeats skill. We replaced both with measures that add new information: skill depth, capacity and past rating (section 4.3). Second, the first prototype let any user see candidate suggestions on any listing. We changed it so that only the listing owner sees suggestions and everyone else can only send a request.

3. Requirements

The requirements follow the IEEE 830 style used in our SRS document (TeamSpark_SRS). Each requirement has an identifier so that a test case can point to the requirement it checks. The tables below show every requirement and whether the prototype implements it.

3.1 Functional requirements

ID	Requirement	Priority	Status in prototype
REQ-TS-01	User registration	High	Not built
REQ-TS-02	User login	High	Not built (single user in prototype)
REQ-TS-03	Profile management (skills)	High	Built: name, skills and levels
REQ-TS-04	Create listing	High	Built
REQ-TS-05	Listing management (edit, close, reopen)	High	Partly built: delete only
REQ-TS-06	Search listings	High	Built: by skill and title
REQ-TS-07	Compatibility scoring	High	Built (revised factors, section 4.3)
REQ-TS-08	Suggested candidates for the owner	High	Built: top 3 with explanation
REQ-TS-09	Send join request	High	Built
REQ-TS-10	Accept or reject request	High	Built
REQ-TS-11	Automatic listing status update when full	Medium	Not built
REQ-TS-12	Notifications	Medium	Not built
REQ-TS-13	Request history	Medium	Built: sent and received requests
REQ-TS-14	Admin: manage listings	Medium	Not built
REQ-TS-15	Admin: manage users	Medium	Not built
REQ-TS-16	Admin reports	Low	Not built

3.2 Non-functional requirements

ID	Requirement	Status in prototype
NFR-TS-01	Performance (responses within 3 seconds)	Met in prototype: all actions are instant
NFR-TS-02	Availability	Not applicable without a server
NFR-TS-03	Security (passwords, tokens)	Not built
NFR-TS-04	Privacy (role-based visibility)	Partly: owner-only suggestions
NFR-TS-05	Reliability (data not lost)	Partly: saved in the browser, with recovery from corrupted data
NFR-TS-06	Usability	Addressed in the interface design; not tested with users
NFR-TS-07	Scalability	Design only (Chapter 4)
NFR-TS-08	Maintainability (separate matching logic)	Built: scoring functions are separate from screen code
NFR-TS-09	Compatibility (desktop and mobile, no app)	Built and tested at phone width

ID	Requirement	Status in prototype
NFR-TS-10	Accessibility	Partly: readable text, visible focus, clear errors
NFR-TS-11	Fairness and transparency (show reasons)	Built: score broken into four bars
NFR-TS-12	Auditability (logging)	Not built

3.3 Users

- **Student (team owner):** posts a listing, sees ranked candidates for it, invites students and answers requests.
- **Student (applicant):** searches listings, sends a request to join, tracks the answer.
- **Administrator (planned):** reviews listings, manages users, sees activity reports.

4. System Design

4.1 UML diagrams

Six diagrams describe the system from different points of view. They are reproduced from the SRS document.

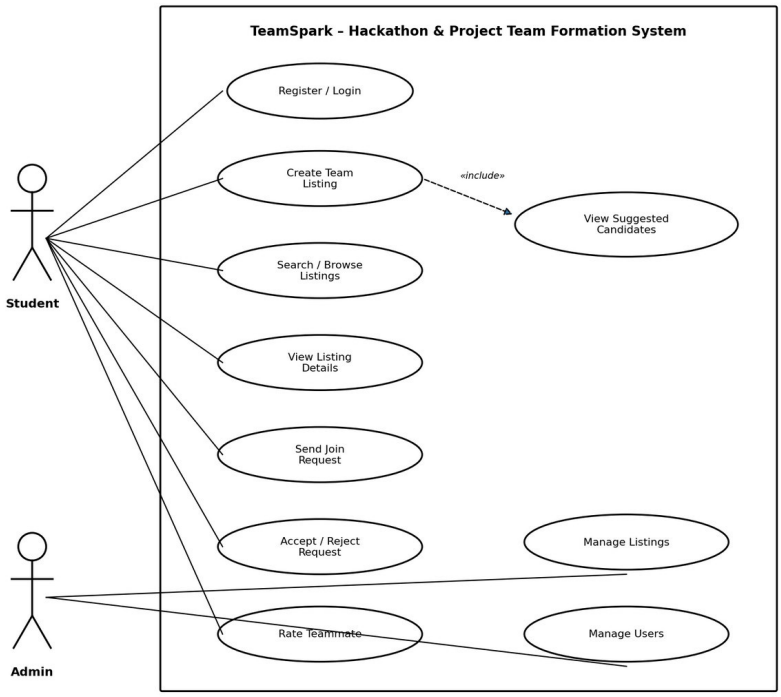


Figure 4.1 Use case diagram: what each user can do. Create Team Listing includes View Suggested Candidates.

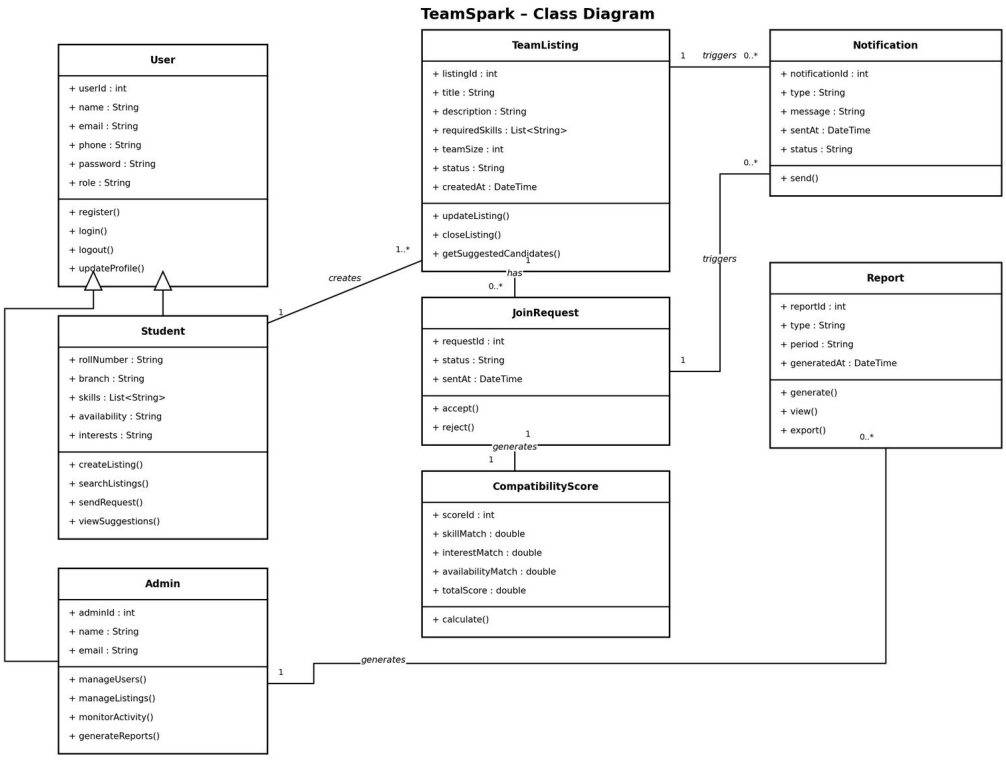


Figure 4.2 Class diagram: classes, attributes, operations and relationships (Student and Admin inherit from User).

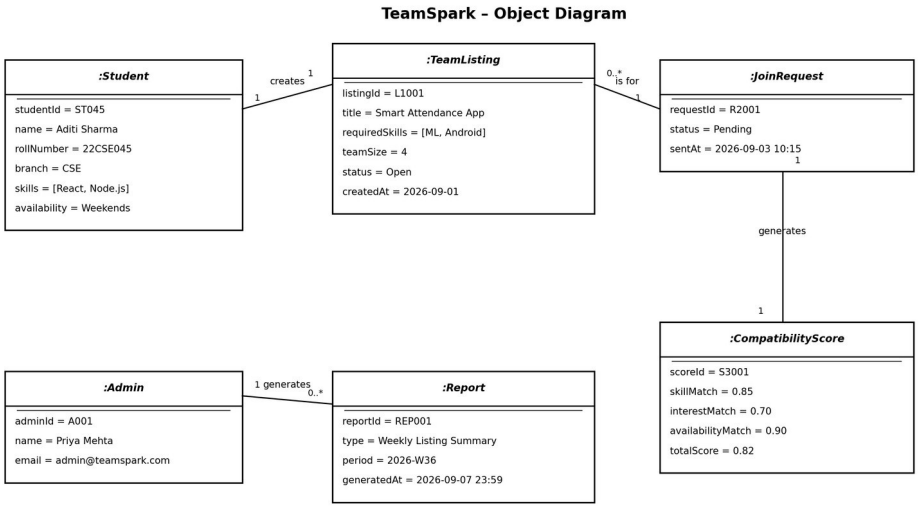


Figure 4.3 Object diagram: a snapshot of the system with example objects.

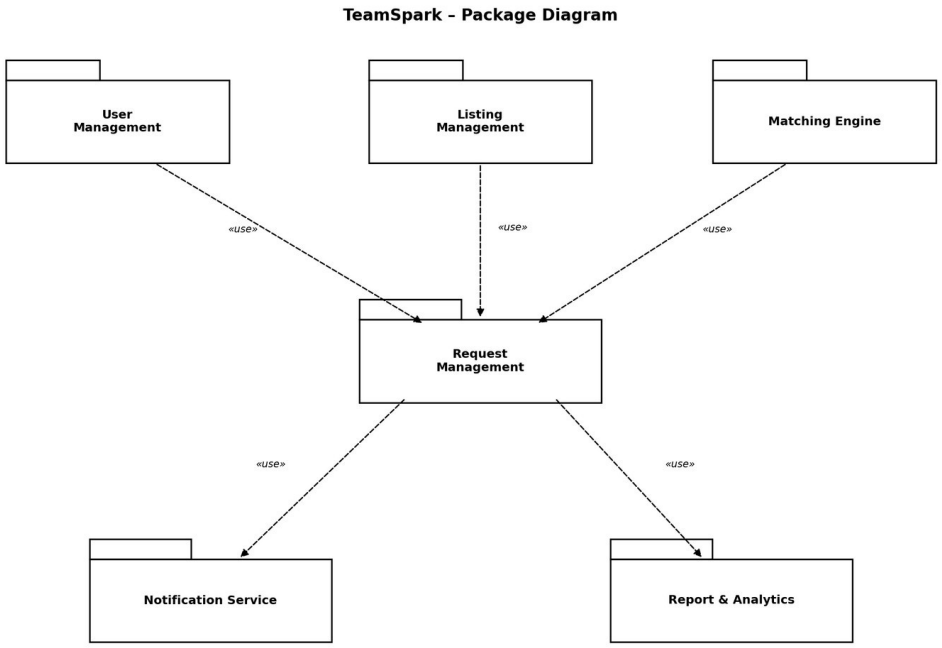


Figure 4.4 Package diagram: grouping of the system into modules and their dependencies.

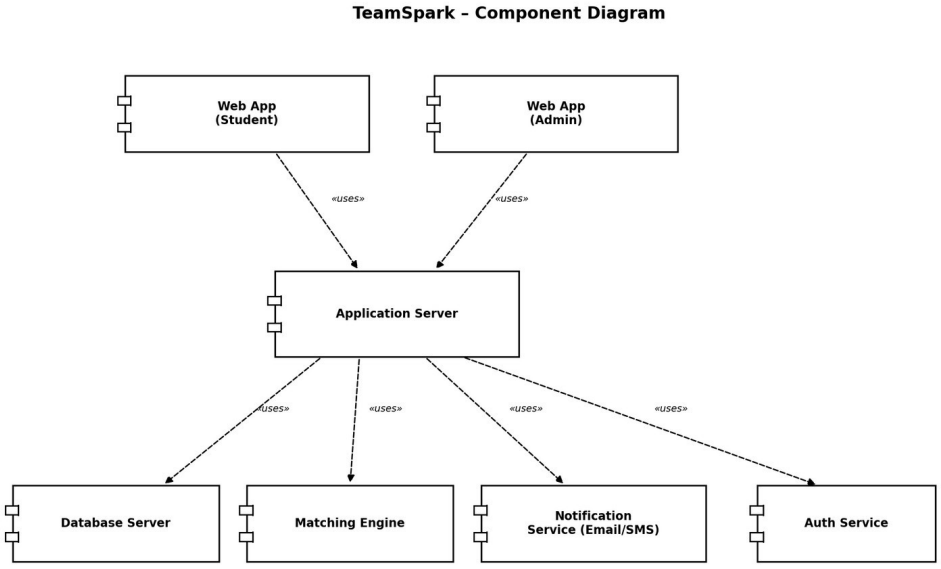


Figure 4.5 Component diagram: software parts of the planned system and how they use each other.

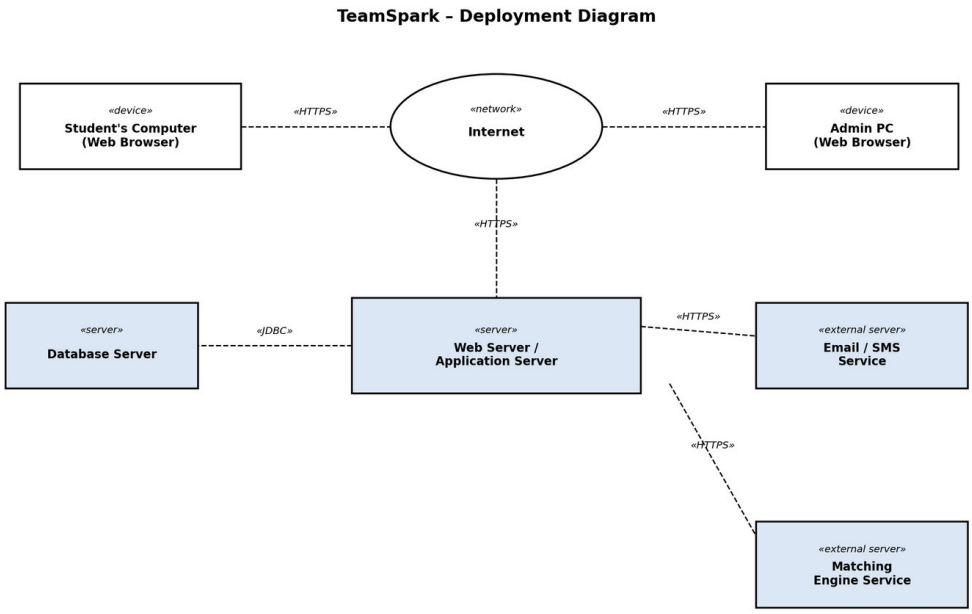


Figure 4.6 Deployment diagram: where each part runs when the system is deployed.

4.2 Note on the Compatibility Score in the diagrams

The class diagram and object diagram show the CompatibilityScore class with the attributes skillMatch, interestMatch and availabilityMatch. These come from the first design. During the project we replaced interest and availability with skill depth, capacity and past rating, as explained in section 2.3. The implemented formula is the one in section 4.3. The two diagrams, and the matching requirement REQ-TS-07 in the SRS, should be read with this change in mind.

4.3 Compatibility Score

For a listing L with a set of required skills and a student S, the score is a weighted sum of four values, each between 0 and 1:

Score = 0.35 × SkillMatch + 0.20 × SkillDepth + 0.25 × Capacity + 0.20 × PastRating

Factor	How it is calculated	Why it is included
SkillMatch (0.35)	Required skills the student has, divided by the number of required skills	The most direct sign that a student can do the work
SkillDepth (0.20)	Average level (1 to 5) of the matched skills, divided by 5	Having a skill at level 5 is not the same as level 1
Capacity (0.25)	max(0, 1 − current projects / 3)	A skilled student with no time cannot help the team
PastRating (0.20)	Average rating from earlier teammates (0 to 1); 0.5 if none yet	Reliability and teamwork seen by others

Worked example. Listing "Smart Attendance App" requires react, node.js and mongodb. Student Arjun Rao has react 4, node.js 5, mongodb 5, one current project and a past rating of 0.95. SkillMatch = 3/3 = 1.00. SkillDepth = (4 + 5 + 5) / 3 / 5 = 0.933. Capacity = 1 − 1/3 = 0.667. PastRating = 0.95. Score = 0.35 × 1.00 + 0.20 × 0.933 + 0.25 × 0.667 + 0.20 × 0.95 = 0.35 + 0.187 + 0.167 + 0.19 = **0.893**, shown as 89%. Test case TC-03 checks exactly this value.

Why this method

- **Content-based matching** compares what a listing needs with what a student has. It works on the first day with no history. Collaborative filtering (people like you joined this team) needs a large amount of past data, which a new system does not have.
- **A weighted formula instead of machine learning** means every score can be explained to the user. Explainability is requirement NFR-TS-11, and it is why the interface shows the four parts of each score.
- **Hybrid decision:** the system ranks, the owner chooses. This keeps the speed of automation and the judgement of a person.

4.4 Planned architecture

The full system is designed as a three-tier web application: a React front end in the browser, a Node.js and Express back end exposing a REST API with JWT-based login, and a PostgreSQL database. The matching logic is a separate module so that the formula can change without touching the rest of the code. The component and deployment diagrams (Figures 4.5 and 4.6) show this arrangement. The prototype in Chapter 5 implements the same logic and screens without the server tier.

4.5 Data flow, data, behaviour and state diagrams

The six diagrams above show structure. The diagrams below show how data moves, what is stored, and how the prototype behaves. They were drawn from the working prototype, so they describe what the code does.

Data flow diagrams

The level 0 diagram treats TeamSpark as one process and shows only what crosses its boundary. The level 1 diagram splits it into four processes. Data stores D1 to D3 are held in the browser; D4 holds the six sample student records.

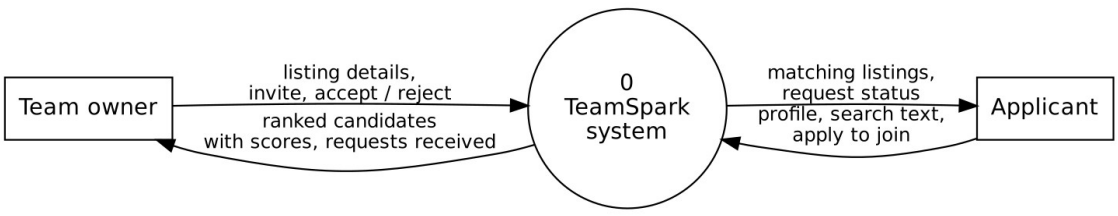


Figure 4.7 Data flow diagram, level 0 (context diagram).

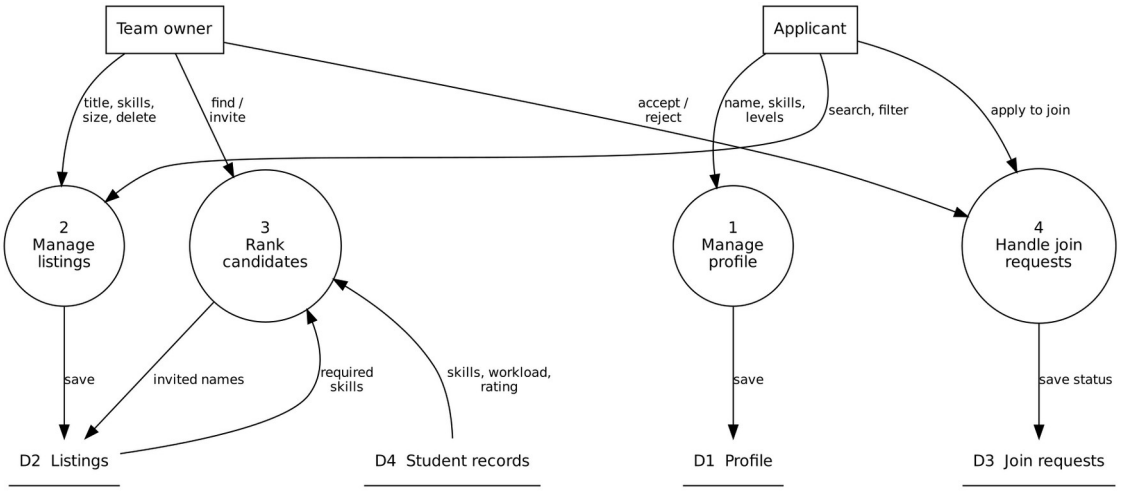


Figure 4.8 Data flow diagram, level 1: four processes, four data stores. Results returned to the owner and applicant are left out to keep the diagram readable.

Entity relationship diagram

A student posts many listings and sends many requests. A listing receives many requests. Skills are linked to students (with a level from 1 to 5) and to listings through two link tables, because a skill can belong to many students and many listings.

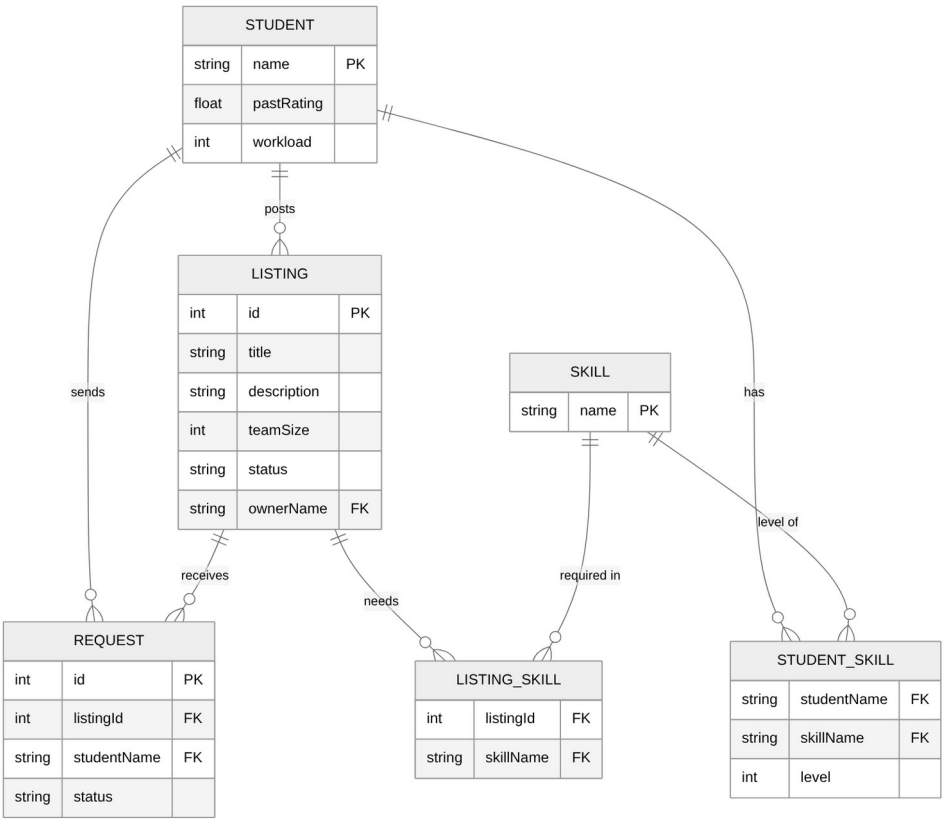


Figure 4.9 Entity relationship diagram.

Sequence diagrams

The first diagram shows the owner finding and inviting candidates. The scoring functions are called once for each student, and the top three are shown. The second shows a join request from the applicant to the owner and back.

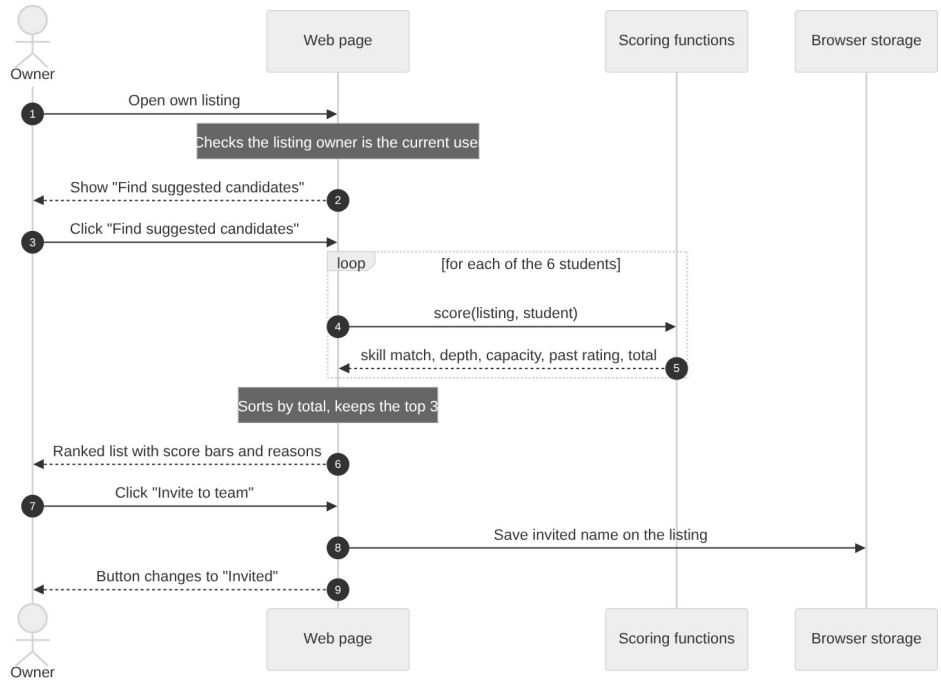


Figure 4.10 Sequence diagram: owner finds suggested candidates and invites one.

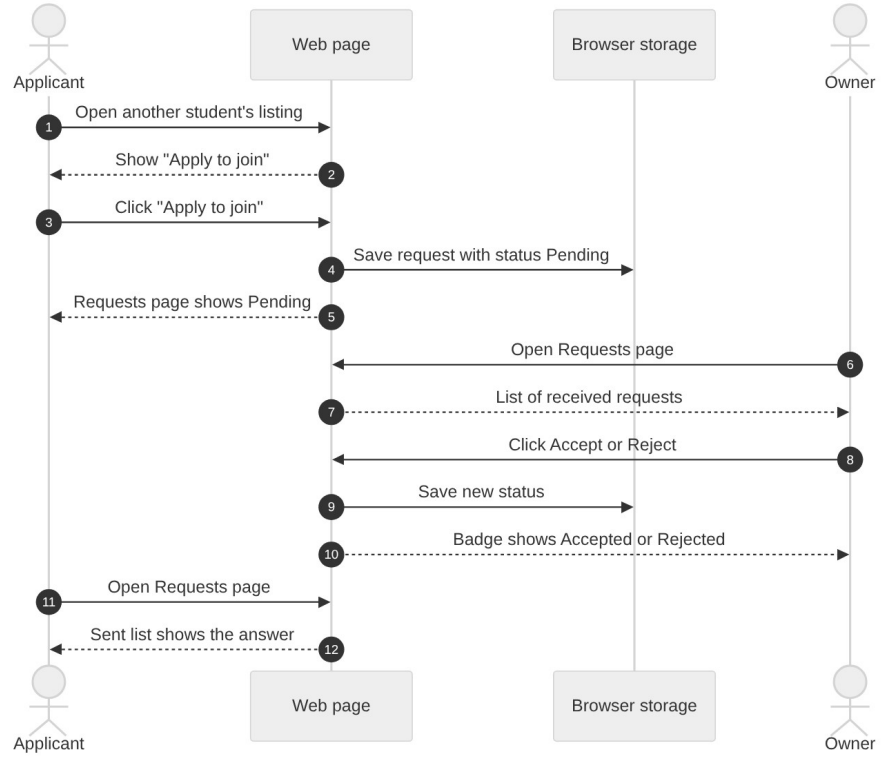


Figure 4.11 Sequence diagram: applicant sends a request, owner answers it.

Activity diagram

This diagram follows the Post listing action, including the check that stops a listing without a title or without any skill.

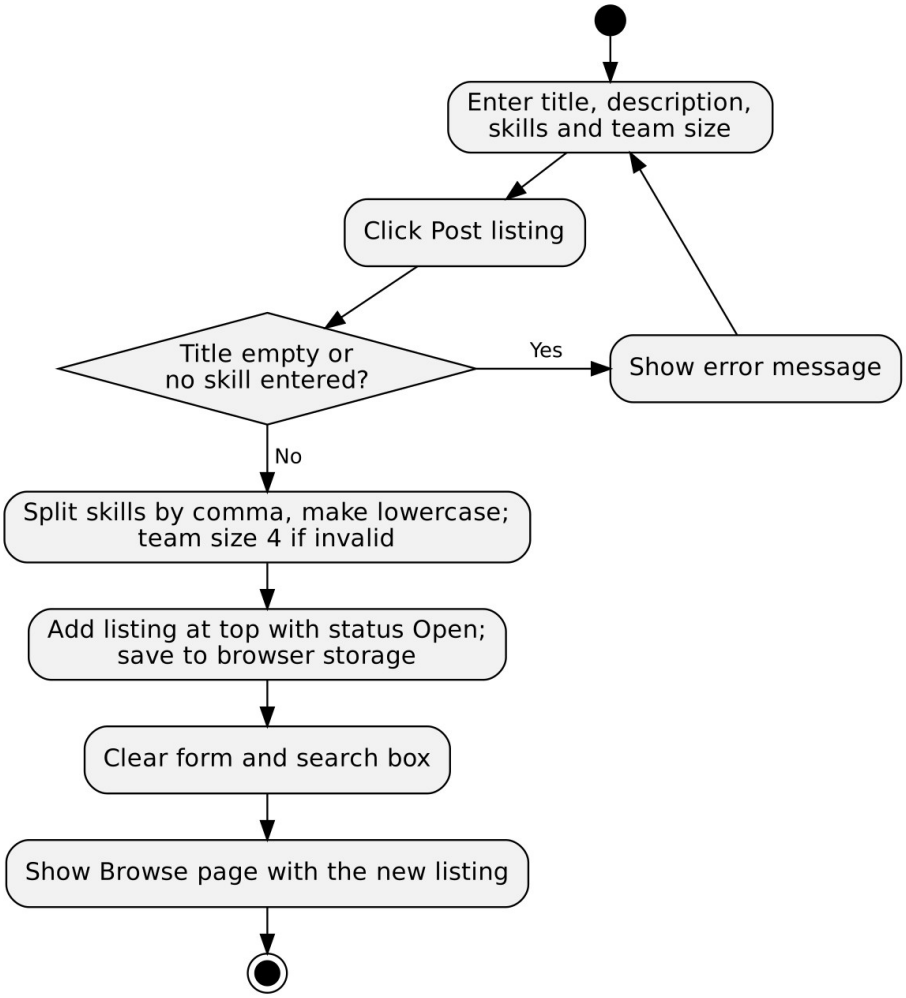


Figure 4.12 Activity diagram: posting a team listing.

State chart diagrams

A join request starts as Pending and the owner moves it to Accepted or Rejected. Deleting the listing removes its requests. The delete button has its own small state chart: the first click arms it, and only a second click within three seconds deletes the listing.

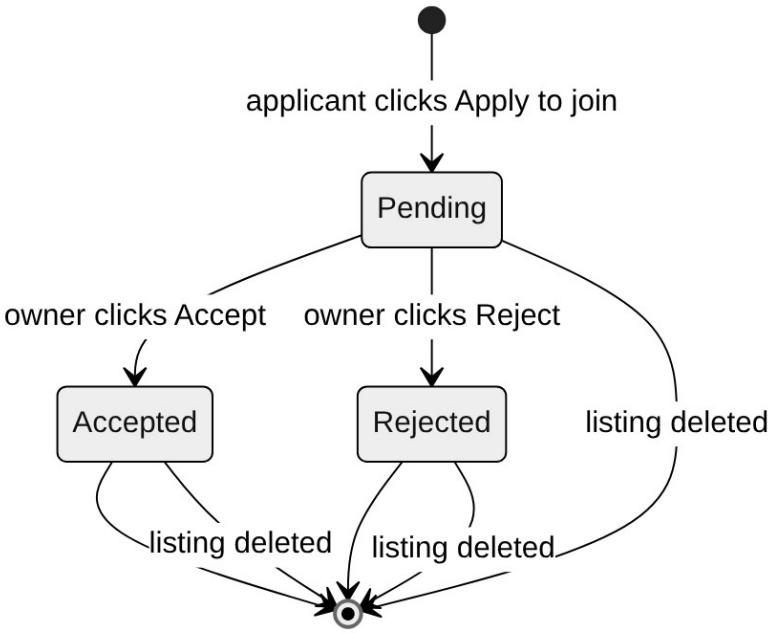


Figure 4.13 State chart: life of a join request.

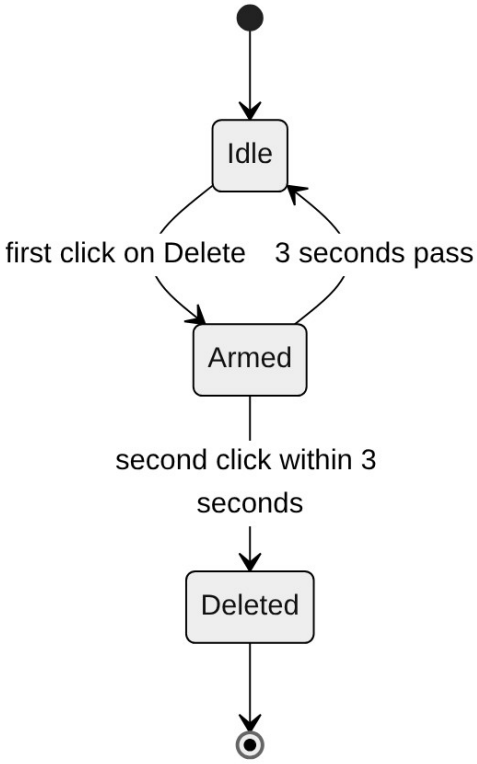


Figure 4.14 State chart: the two-click delete button.

5. Implementation

5.1 Technology of the prototype

The prototype is a single web page written in HTML, CSS and JavaScript with no framework and no server. Data is saved in the browser (localStorage) so that listings, requests and the profile survive a page reload. It was built this way so that it can be opened and demonstrated anywhere, and so that the matching logic can be tested directly. The size of the page script is about 296 lines of JavaScript.

5.2 Code structure

Functions	Purpose
skillMatch, skillDepth, workloadCap, score	Matching logic: the Compatibility Score
renderGrid, setFilter, renderFilters	Show listings, search by skill, skill filter buttons
openListing, closePanel	Side panel with details; shows suggestions button to the owner and apply button to others
findMatches, inviteCandidate	Rank the six sample students, show the top three with reasons; invite
applyToJoin, renderRequests, setStatus	Join requests and the owner response
createListing, confirmDelete, deleteListing	Post a listing with validation; two-click delete; remove linked requests
loadProfileForm, saveProfile	Profile with skills and levels (kept between 1 and 5)
loadState, saveState, defaultState	Save to browser storage; recover from missing or corrupted data

The scoring functions have no dependence on the screen. They take a listing and a student and return numbers. This separation is what makes them easy to test (Chapter 7) and follows NFR-TS-08.

5.3 Owner and applicant rules

The same listing looks different to different users, and this is enforced in code (function openListing):

Situation	What the user sees	Reason
User owns the listing	"Find suggested candidates" button; no apply button	Only the owner chooses who joins
User does not own it, has not applied	"Apply to join" button	Others can only ask
User does not own it, has applied	Disabled "Request sent" button	Prevents duplicate requests

5.4 Screens

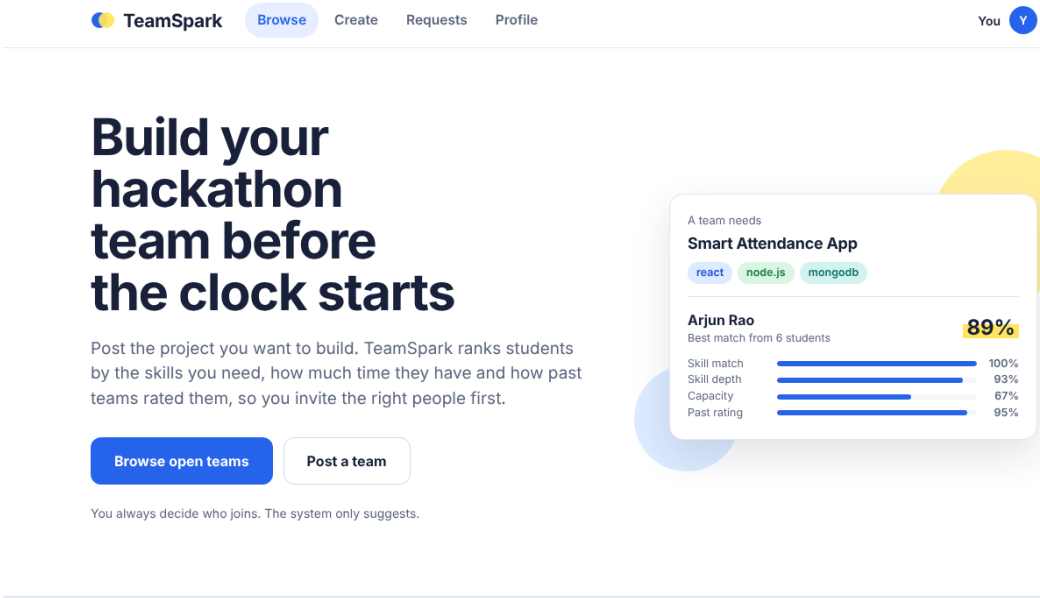


Figure 5.1 Home page: headline, an example of a ranked match, and the two main actions.

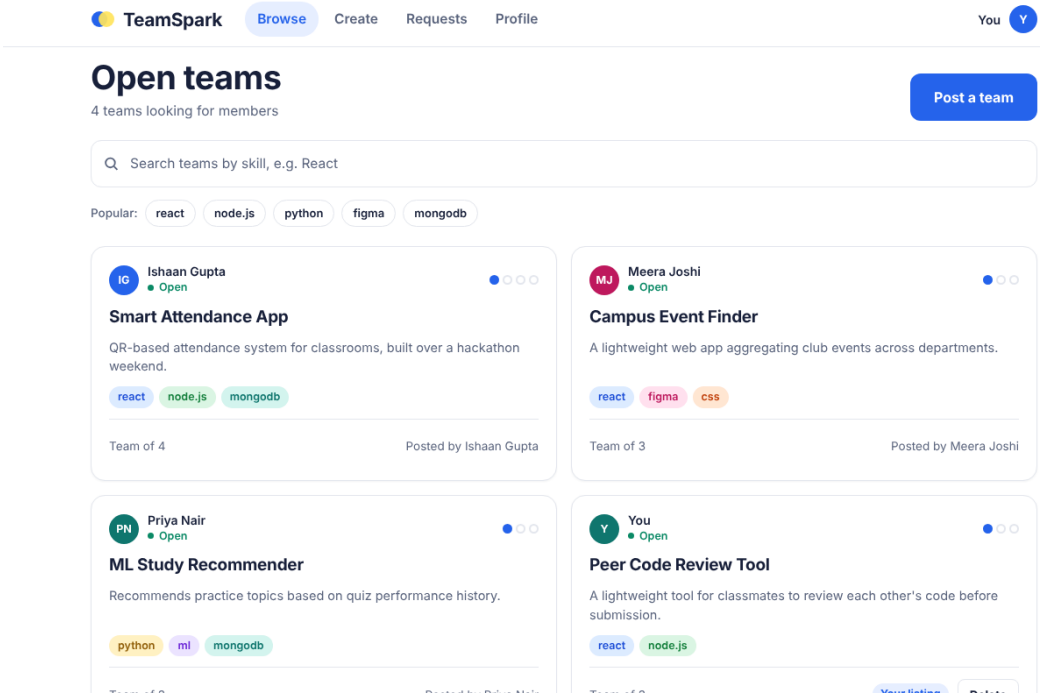


Figure 5.2 Open teams: search, skill filters and team cards with colour-coded skills.

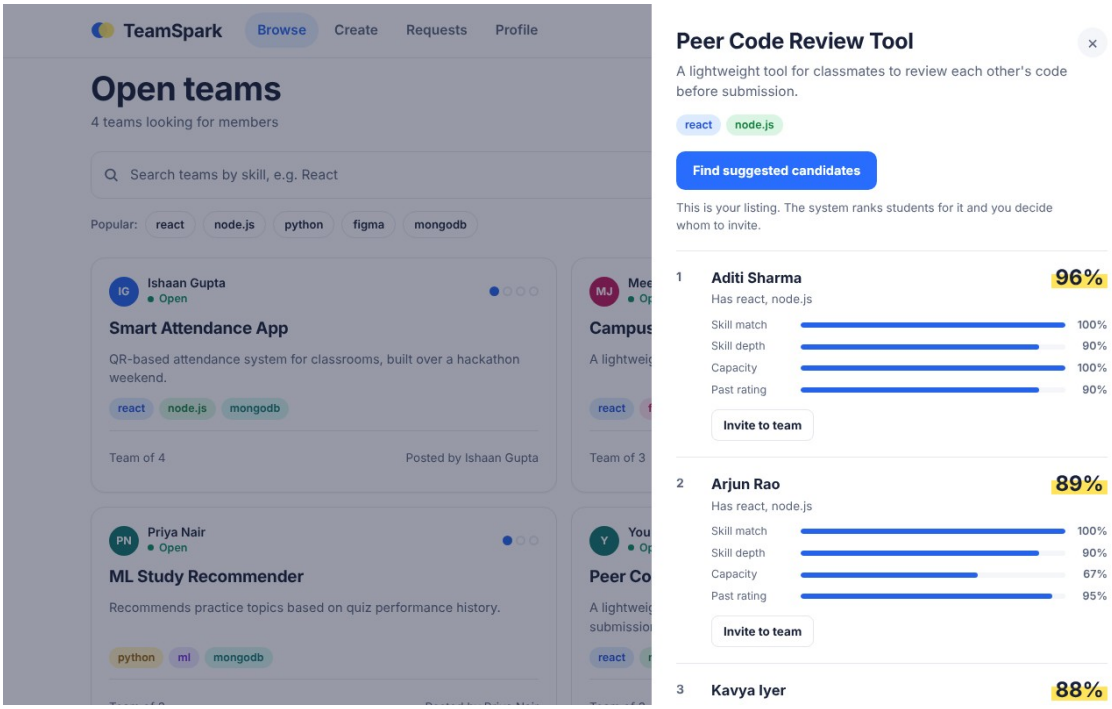


Figure 5.3 Suggested candidates for the owner: ranked students with the score and its four parts.

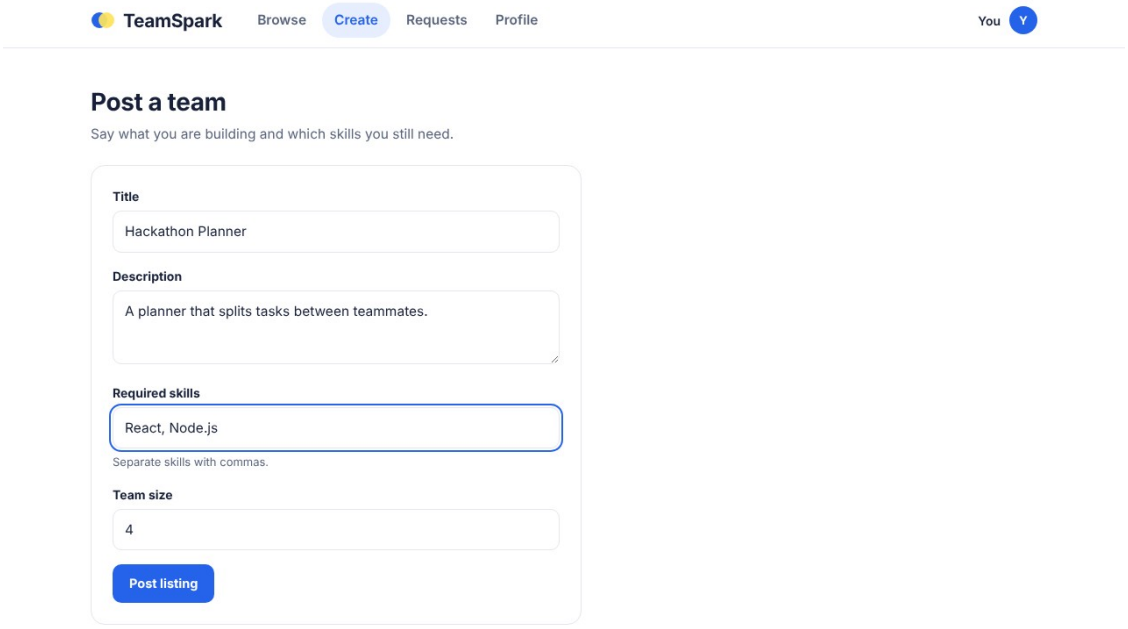


Figure 5.4 Post a team form.

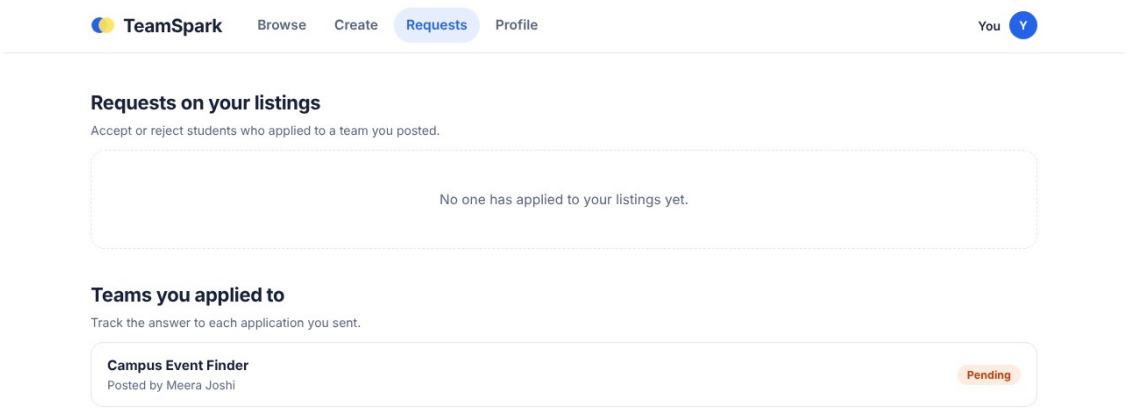


Figure 5.5 Requests: replies to the owner and the status of applications sent.

5.5 Limitations of the prototype

- Single user: there is no login, and data is stored only in the browser of the person using it.
- The six students used for ranking are sample data inside the page. In the full system they would come from registered profiles in the database.
- Features listed as not built in Chapter 3 are future work.

6. Quality Assurance and DevOps

6.1 Software quality

Software quality was handled through the following activities:

- **Reviews:** each requirement and design decision was reviewed by both team members, which led to the scoring and access-rule changes described earlier.
- **Requirement traceability:** every test case names the requirement it checks (Appendix A), so no important requirement is left untested without us knowing.
- **Testing at two levels:** logic tests for correctness, and browser tests for what the user sees.
- **Measurement:** code coverage and cyclomatic complexity (Chapter 7) give numbers for how well the code is exercised and how complex it is.

On the Capability Maturity Model our process is best described as Level 1 to 2 (initial, moving towards repeatable): the work is planned and documented, but there is no organisation-wide process.

6.2 DevOps practice

DevOps aims to build, test and release software automatically. Two parts apply to this project:

- **Automated tests as a single command:** npm run test:all runs every test with no manual steps.
- **Continuous integration:** a GitHub Actions workflow file (.github/workflows/tests.yml) is included. When the code is pushed to GitHub, it installs the tools and runs the logic tests, the coverage report and the browser tests on every push. The workflow is ready but has not yet run, because the code is not yet in a GitHub repository.

Releasing the website would need a host such as GitHub Pages. Docker and cloud deployment are part of the full system and are future work.

7. Testing

7.1 Approach and tools

Testing was designed in two complementary ways. **Black-box testing** treats the program as a closed box: we choose inputs from the requirements and check the outputs, without using knowledge of the code. **White-box testing** uses the code: we draw the flow of the program, find every path through it and make sure tests exercise them.

Tool	Use in this project
Jest 29	Runs the real application script with a simulated page and storage. 13 logic tests (TC-01 to TC-13). Also produces the coverage report.
Playwright 1.56	Opens the site in a real Chromium browser, types and clicks like a user. 7 tests (UI-01 to UI-07).

Selenium and Cypress are alternatives that also automate a browser. Playwright was chosen because it needs no separate driver and its tests are short and reliable.

7.2 Black-box techniques applied

Technique	Idea	Where we used it
Equivalence partitioning	Divide inputs into groups that the program should treat the same, and test one value from each group.	Listing form: valid / empty title / no skills (TC-08, TC-12, UI-02). Search: matching / non-matching text (TC-04, UI-01). Skills matched: all / some / none (TC-01).
Boundary value analysis	Test at and next to the edges of each group, where errors cluster.	Skill level 0, 1, 5, 6 and text (TC-11). Workload 0, 1, 3 (the point where capacity reaches 0) and 5 (TC-02).
Decision table	List every combination of conditions and check the action for each.	Owner / applied combinations (TC-06, UI-03, UI-05); see Table 7.2.
State transition testing	Check the allowed changes of an object between states.	A request goes Pending to Accepted or Pending to Rejected (TC-07, TC-13); accepting twice does not duplicate membership.
Use-case (scenario) testing	Run a complete user story from start to finish.	Post a team, find candidates, invite (UI-04); apply and see Pending (UI-05); delete a team (UI-06).
Error guessing	Try situations where a developer is likely to have made a mistake.	Corrupted saved data (TC-10); skill level typed as text or zero (TC-11).

Equivalence partitions and boundary values: skill level in the profile

A skill level must be a number from 1 to 5. The partitions are: below 1 (invalid, too low), 1 to 5 (valid), above 5 (invalid, too high) and not a number (invalid). The boundary values are 0, 1, 5 and 6.

Input	Partition	Expected result (checked in TC-11)
0	Too low (boundary just below 1)	Becomes 1
1	Valid (lower boundary)	Stays 1
5	Valid (upper boundary)	Stays 5
6	Too high (boundary just above 5)	Becomes 5
abc	Not a number	Becomes 3, the default level

Decision table: access rules for a listing

Table 7.2 Conditions and actions when a user opens a listing

Rule	Owner?	Already applied?	Action shown
R1	Yes	Not applicable	Find suggested candidates; no apply button (TC-06, UI-03)
R2	No	No	Apply to join, enabled (TC-06, UI-03)
R3	No	Yes	Request sent, disabled (UI-05)

State transition: join request

A request is created in the state Pending. The owner moves it to Accepted, which also adds the team to the student's memberships, or to Rejected, which changes nothing else. Once a request has left Pending, the interface no longer offers the Accept and Reject buttons. TC-07 checks Pending then Accepted. TC-13 checks Rejected, and checks that accepting the same listing twice does not create a duplicate membership.

7.3 White-box techniques applied

White-box testing starts from the structure of the code. The following terms are used.

- **Flow graph (control flow graph):** nodes are statements or blocks, edges show which can follow which.
- **DD-path (decision-to-decision path) graph:** the flow graph with every straight run of statements, which has no decision inside it, merged into a single node. What remains are the decision points and the chains between them. This is the form used for path testing (Jorgensen).
- **Cyclomatic complexity V(G) = E − N + 2**, where E is the number of edges and N the number of nodes. It equals the number of independent paths, and also the number of decisions plus one. Each && and || is counted as a decision.
- **Basis path testing:** choose V(G) independent paths so that every edge is covered at least once, and write one test per path.

7.3.1 Function createListing (posting a team)

The function reads the form, rejects the input if the title is empty or no skills are given, and otherwise adds the listing, saves it, clears the form and shows the list. The condition !title || !skills.length is split into two decision nodes because || evaluates the second part only if the first is false.

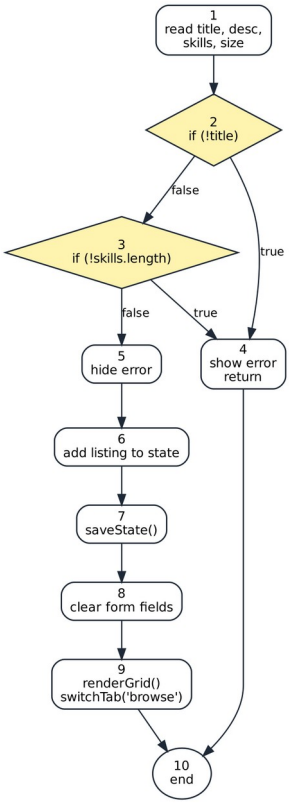


Figure 7.1 Flow graph (N = 10, E = 11)

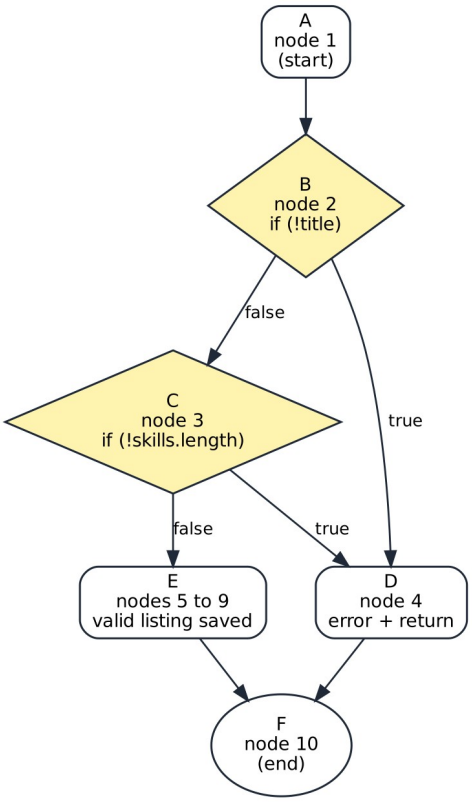


Figure 7.2 DD-path graph (N = 6, E = 7)

From the flow graph: V(G) = 11 − 10 + 2 = 3. From the DD-path graph: V(G) = 7 − 6 + 2 = 3. There are two decisions (|| counted once, plus the if), so decisions + 1 = 3. All three agree. Hence three basis paths are needed:

Path	DD-path sequence	Input condition	Test case
P1	A, B, D, F	Title is empty	TC-08 (second half), UI-02
P2	A, B, C, D, F	Title given, no skills	TC-12
P3	A, B, C, E, F	Title and skills valid	TC-08 (first half), UI-02

7.3.2 Function setStatus (owner answers a request)

The function sets the new status. If the status is Accepted and the listing is not yet in the student's memberships, it adds it. Then it saves and refreshes the page. Again the compound condition status === 'Accepted' && !memberOf.includes(…) gives two decision nodes.

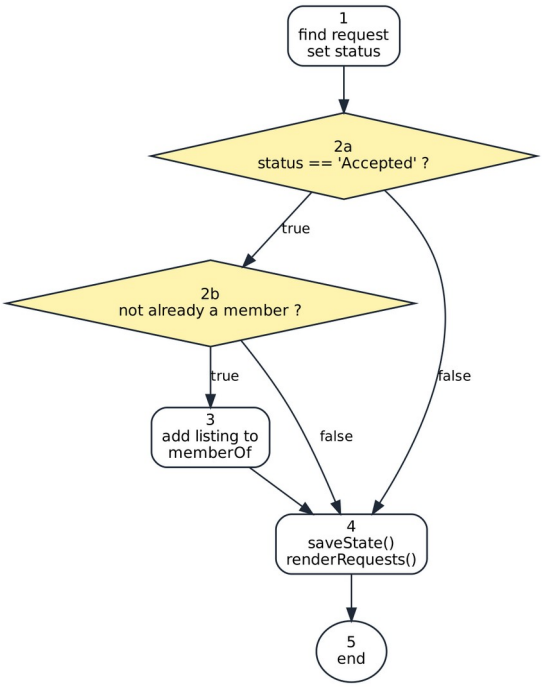


Figure 7.3 Flow graph (N = 6, E = 7)

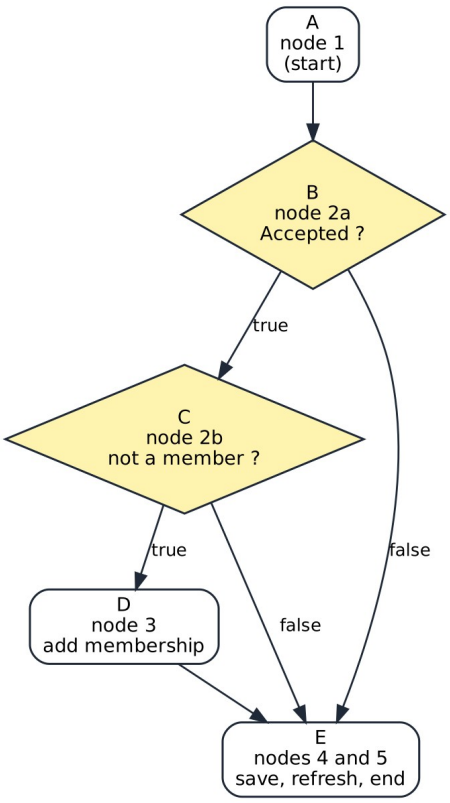


Figure 7.4 DD-path graph (N = 5, E = 6)

V(G) = 7 − 6 + 2 = 3 (flow graph) and 6 − 5 + 2 = 3 (DD-path graph). The three basis paths are:

Path	DD-path sequence	Input condition	Test case
P1	A, B, E	Status is Rejected	TC-13 (first half)
P2	A, B, C, D, E	Accepted, not yet a member	TC-07
P3	A, B, C, E	Accepted, already a member	TC-13 (second half)

Our first set of tests exercised only path P2. Analysing the DD-path graph showed that paths P1 and P3 were untested, so TC-13 was added. This is the practical value of path analysis: it reveals branches that look covered because the function was called, but whose other outcomes were never checked.

7.3.3 Cyclomatic complexity of other functions

Function	V(G)	Reading
createListing	3	Three paths, all tested (section 7.3.1)
setStatus	3	Three paths, all tested (section 7.3.2)
skillMatch	2	One decision (empty required list), both outcomes tested in TC-01 and the ranking tests
skillDepth	2	One decision (no matching skill), tested through the score tests
workloadCap	1	No decision (a single maximum), tested for 0, 1, 3 and 5 projects in TC-02

All values are well below 10, the usual limit above which a function is considered hard to test and maintain. They were counted by hand from the code using the rule above.

7.4 Coverage measured

Coverage measures how much of the code the tests actually run. Jest with V8 coverage measured the application script while the 13 logic tests ran:

Measure	Result	Meaning
Statement coverage	85.89%	Share of statements executed at least once
Branch coverage	79.64%	Share of decision outcomes (true and false) taken
Function coverage	81.25%	Share of functions called at least once

The part not covered is mainly the code that draws the screen (request lists, closing the side panel), which needs a browser. The Playwright tests exercise those parts, but the figures above come only from the logic tests, so they understate the total.

7.5 Results

20 test cases were run: 20 passed and 0 failed. The full table, with the technique used for each test, is in Appendix A.

7.6 Defect found and fixed

Item	Detail
Found by	Boundary value analysis of the skill level (TC-11)
Problem	A skill level of 0 was turned into 3 instead of 1. The code used parseInt(v) || 3, and the number 0 counts as "false" in JavaScript, so the default was applied.
Impact	Low: a user who wrote level 0 got a higher level than intended.
Fix	Check explicitly whether the input is a number (Number.isNaN) and only then apply the default.
Status	Fixed; TC-11 passes for 0, 1, 5, 6 and text.

During browser testing, an earlier version of the page also scrolled sideways on a very small phone screen because the navigation bar was 2 pixels too wide. It was fixed by reducing the padding on small screens, and UI-07 now guards against it.

7.7 Limits of the testing

- Browser tests ran in Chromium only. The sample data is small (six students), so the ranking tests check the formula and the order, not behaviour at scale.
- Features that are not built (REQ-TS-01, 02, 11 to 16) are not tested. Load, security and accessibility testing need separate tools and were outside this round.

8. Conclusion and Future Work

8.1 Conclusion

TeamSpark addresses a real problem in student hackathons: forming balanced teams without relying on personal contacts. The project followed the software engineering process from problem statement and SRS through design, a working prototype and testing. The Compatibility Score is explainable, built from four measurable factors, and used in a hybrid way so that the final decision stays with a person. The prototype implements the core requirements that do not need a server, and 20 of 20 automated tests pass, with 85.89% statement coverage of the logic code.

8.2 Future work

- Registration, login and a shared database so that several users can use the system together (REQ-TS-01, 02).
- Edit and close a listing, and close it automatically when the team is full (REQ-TS-05, 11).
- Notifications for new requests and answers (REQ-TS-12).
- Administrator functions: manage listings and users, activity reports (REQ-TS-14 to 16).
- Collect real ratings after a hackathon so that PastRating comes from data, and re-check the weights of the formula against how teams actually performed.
- Push the code to GitHub so the included continuous integration workflow runs on every change.

References

- R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill, 2019.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- W. Boggs and M. Boggs, Mastering UML with Rational Rose, SYBEX, 1999.
- P. C. Jorgensen, Software Testing: A Craftsman's Approach, CRC Press (decision-to-decision paths and path testing).
- G. J. Myers, The Art of Software Testing, Wiley (equivalence partitioning, boundary value analysis).
- M. Walls, Building a DevOps Culture, O'Reilly, 2013.
- Jest documentation, https://jestjs.io and Playwright documentation, https://playwright.dev

Appendix A: Test Case Table

Type: BB = black-box, WB = white-box (designed from a path of the code). TC = logic test (Jest), UI = browser test (Playwright).

Test ID	SRS requirement	Type	Technique	What is done (input)	Expected result	Status
TC-01	REQ-TS-07	BB	Equivalence partitioning	skillMatch when student has 2 of the 3 required skills	0.6667	Pass
TC-02	REQ-TS-07	BB	Boundary value analysis	workloadCap for 0, 1, 3 and 5 current projects	1.0, 0.6667, 0 and 0 (never negative)	Pass
TC-03	REQ-TS-07	BB	Functional (formula check)	total Compatibility Score of Arjun Rao for "Smart Attendance App"	0.8933	Pass
TC-04	REQ-TS-06	BB	Equivalence partitioning	search "python", then " REACT " (capitals and spaces)	1 match for python, 3 matches for react	Pass
TC-05	REQ-TS-08	BB	Functional (ranking)	findMatches for "Smart Attendance App"	top 3 are Arjun, Kavya, Aditi in this order; others not shown	Pass
TC-06	REQ-TS-08, 09	BB	Decision table	open own listing, then someone else's listing	owner sees "Find suggested candidates" only; other user sees "Apply to join" only	Pass
TC-07	REQ-TS-09, 10	BB + WB	State transition, path P2 of setStatus	apply to a listing, then owner accepts	request is Pending first, then Accepted and the listing is added to memberOf	Pass
TC-08	REQ-TS-04	BB + WB	Equivalence partitioning, paths P1 and P3 of createListing	post a valid listing, then post one with an empty title	valid one is added first with owner "You" and lowercase skills; empty title is rejected with an error	Pass
TC-09	REQ-TS-05	BB	Functional (cause and effect)	delete a listing that has a join request	listing and its request are both removed	Pass
TC-10	NFR-TS-05	BB	Error guessing	create a listing and reload; then load corrupted stored data	listing is restored after reload; corrupted data falls back to the 4 sample listings without crashing	Pass
TC-11	REQ-TS-03	BB	Boundary value analysis	save skill levels 0, 1, 5, 6 and "abc" (boundary values)	levels become 1, 1, 5, 5 and 3 (kept inside 1 to 5; non-number defaults to 3)	Pass
TC-12	REQ-TS-04	BB + WB	Equivalence partitioning, path P2 of createListing	post a listing that has a title but no skills	rejected with an error, nothing added	Pass
TC-13	REQ-TS-10	BB + WB	State transition, paths P1 and P3 of setStatus	reject a request; then accept the same listing twice	rejecting adds no membership; accepting twice keeps one membership (no duplicate)	Pass

Test ID	SRS requirement	Type	Technique	What is done (input)	Expected result	Status
UI-01	REQ-TS-06	BB	Equivalence partitioning	type "python" in the search box, then type "cobol"	only ML Study Recommender for python; "No listings match" message for cobol	Pass
UI-02	REQ-TS-04	BB	Equivalence partitioning	fill the form and click Post listing; then submit with an empty title	new listing appears in Browse (5 teams); empty title shows an error and adds nothing	Pass
UI-03	REQ-TS-08, 09	BB	Decision table	open your own listing, then another student's listing	your listing shows only "Find suggested candidates"; the other shows only "Apply to join"	Pass
UI-04	REQ-TS-08	BB	Use-case (scenario) testing	post a listing needing react, node.js, mongodb; open it and click Find suggested candidates; invite the first candidate	3 candidates, Arjun Rao first with 89%; Invite button turns into disabled "Invited"	Pass
UI-05	REQ-TS-09	BB	Decision table, scenario	open Smart Attendance App and click Apply to join; then open the same listing again	Requests page shows the application as Pending; reopening the listing shows a disabled "Request sent" button	Pass
UI-06	REQ-TS-05	BB	Use-case (scenario) testing	click Delete on your own listing twice	first click asks "Click again to confirm"; second click removes the listing (3 teams left)	Pass
UI-07	NFR-TS-09	BB	Compatibility (screen size)	open the site at phone width (375 x 700 px)	listings visible and the page does not scroll sideways	Pass

Appendix B: Running the Project and Tests

Files

- teamspark.html: the website (open it in a browser).
- teamspark.test.js: logic tests (Jest). e2e/teamspark.spec.js: browser tests (Playwright).
- package.json and playwright.config.js: tool setup. .github/workflows/tests.yml: continuous integration workflow.

Commands

Put the files in one folder, then run:

- npm install installs Jest and Playwright (once).
- npx playwright install chromium downloads the test browser (once).
- npm test runs the logic tests.
- npm run test:coverage runs the logic tests and prints the coverage table.
- npm run test:e2e runs the browser tests (add --headed to watch the browser).
- npm run test:all runs everything.

Demo video

A two-minute screen recording with captions, TeamSpark_Demo.mp4, shows posting a team, searching, applying, seeing ranked candidates, inviting, editing the profile and deleting a team.